第22章 基礎圖演算法：詳解

22-E01 表示法與零權重

推導

一條無向邊在兩個端點各佔一項，因此9條邊有18項；6個表頭另算。BFS 初始化6點，各點最多入列一次，全部可達時掃18個鄰接項，時間 O(n+m) 而非 O(nm)。完整6×6 matrix 有36格，一般 n 點為 $\Theta(n^2)$ 空間。

若 u→v 權0，把 A[u,v]=0 同時當無邊會失去連通資訊。例如唯一邊 u→v 權0，若讀為無邊就會把可達誤判不可達。可以存 pair(exists,weight)，或用非數值標記 NIL 表示不存在；合法零權仍存數值0。

結論與檢核

18個鄰接項，list BFS 為 O(n+m)，matrix 空間 $\Theta(n^2)$。零權與不存在必須可區分。若採∞表示無邊，也須確認∞不是允許的實際邊權。

22-E02 六點圖的 BFS

推導

初始化 queue=[d]、d[d]=0，其餘∞。取d後發現b、e，queue=[b,e]，兩點距離1且π均為d。取b後發現c，queue=[e,c]，d[c]=2、π[c]=b。取e時d已發現，queue=[c]。取c後發現a，queue=[a]，d[a]=3、π[a]=c。取a時b、c均已發現，queue=[]。

最後按a,b,c,d,e,f排列，距離為(3,1,2,0,1,∞)，π=(c,d,b,NIL,d,NIL)。f未發現，因為f→e只允許從f走到e；不能逆向使用。

有向邊a→b兩端距離3與1，相差2。BFS保證$d[b] \le d[a]+1$，即$1 \le 4$；沒有要求$d[a] \le d[b]+1$，因為b→a不在E。實際有b→c→a長2，並非一條反向單邊。

結論與檢核

dequeue順序d,b,e,c,a，距離非遞減0,1,1,2,3。每個有限π鏈逐步減1，到a的最短鏈是d→b→c→a。$|d[u]-d[v]| \le 1$的雙向結論限無向邊。

22-E03 SCC 與轉置的跨界

推導

第一趟遞迴a→b→c→d，事件a入1、b入2、c入3、d入4、d出5、c出6、b出7、a出8。因此a=1/8、b=2/7、c=3/6、d=4/5，完成序d,c,b,a，反序候選根a,b,c,d。

轉置表a:[b]、b:[a]、c:[b]、d:[c]。第二趟從a收{a,b}；b已黑跳過。從c掃到已黑b，只收{c}。從d掃到已黑c，只收{d}，共三類。

第22章 基礎圖演算法：詳解 （續）

若任意先從d起跑，會沿d→c→b→a到達全部四點，卻不能說它們互相可達。例如原圖d無出邊，不能回c。轉置保留的是「互相可達等價類」，不保證任一起點只走在一類；正確的先大完成值與已黑邊界才保證第二趟不跨到未處理類。

結論與檢核

SCC為{a,b}、{c}、{d}；原縮合圖{a,b}→{c}→{d}，其最大完成值8>6>5。任意GT DFS可合併這三類；這正是省略根順序的反例。

22-E04 同一圖的 DFS 邊型

推導

依事件a入1、b入2、c入3、c出4、b出5、a出6、d入7、e入8、e出9、d出10、f入11、f出12。π[a]=π[d]=π[f]=NIL，π[b]=a、π[c]=b、π[e]=d。

按掃描順序：a→b為T，b→c為T，c→a時a灰為B；返回a後掃a→c，c黑且tin[a]<tin[c]，為F。d→b時b已完成且非祖先為C；d→e為T；e→d時d灰為B；f→e時e已黑且與f區間分離為C。

a→c沒有觸發發現c，所以不是tree；但c的區間[3,4]嚴格包含於a的[1,6]，c是a真後代，所以不是cross。tree和forward都符合嚴格巢狀同一時間關係，需再讀π[v]=u或記錄實際發現邊才能區分。

結論與檢核

8邊共3T、2B、1F、2C。時間1到12沒有重覆，forest三根a,d,f。只有後代時間不代表直接父子；a→c正好展示這個差別。

22-E05 拓撲序的合法性與殘餘

推導

a,b,c,d與b,a,c,d均讓a與b排在c前、c在d前，故合法。a,c,b,d讓b→c的起點排在終點後，故不合法，雖然c→d仍正確也不能補救。

原圖初始indegree=(0,0,2,1)、queue=[a,b]。輸出a後c變1，queue=[b]；輸出b後c變0，queue=[c]；輸出c後d變0，queue=[d]；輸出d後空，全部四點輸出。

加d→b,d→z後，入度a0,b1,c2,d1,z1。初始queue=[a]；輸出a使c入度2→1，queue即空。殘餘{b,c,d,z}，cycle為b→c→d→b。z是d的下游，無出邊，並不位於任何cycle。

結論與檢核

Kahn只輸出a就正確判定有cycle。殘餘非空且每點都有殘餘入邊，沿入邊可找到某個cycle；這不等於殘餘每點本身都在cycle上。

第22章 基礎圖演算法：詳解 （續）

22-E06 白路徑與一般可達

推導

事件為a入1、b入2、b出3、c入4、c出5、a出6。c入前a為灰、b為黑、c為白，所以白點集合只有{c}。路徑c→b不是全白路徑，因b已黑。

b的區間[2,3]與c的[4,5]分離，π[b]=π[c]=a；兩者是兄弟而非祖孫。c掃到b時不會再遞迴，舊發現不能移到新子樹。white-path theorem要求在根u即將被發現的同一切點，整條路徑仍白，不能換成「原圖裡存在路徑」。

結論與檢核

c可達b而b不是c後代，完全符合定理。真正白路徑只有c到自身的零長路徑，對應c是自身的非嚴格後代；使用「真後代」時則需排除v=u。

22-E07 最短路徑數與位元成本

推導

從0發現1、2，兩者距離1、ways各1。處理1使3、4的距離成2、ways各加1；處理2時3、4已發現但d仍為d[2]+1，還要各加1，因此ways[3]=ways[4]=2。處理3首次發現5，d[5]=3、ways[5]=2；處理4再次滿足距離關係，再加2，最後ways[5]=4。

所以d=(0,1,1,2,2,3)、ways=(1,1,1,2,2,4)。若只在首次發現時計數，後來同層前驅的貢獻會被漏掉，這個圖甚至會讓3與4先各錯成1，進而把5錯算成1。

有L層、每層兩點，從單一source連到第一層，逐層完全連接，再接單一sink，source到sink最短路徑為2^L，n=2L+2，故數值為2^Θ(n)，其二進位長度Θ(n)。演算法使用O(n+m)次整數加法，但位元模型下每次加法可需O(n)，直接實作可用O(n(n+m))位元操作作為上界；不能因加法次數線性就忽略數字長度。

結論與檢核

同一距離層的點處理順序不影響最終ways，因每條最短前驅邊都納入一次；先前一層在下一層出列前必已全部出列。線性界是unit-cost arithmetic或固定可容納整數的模型，若使用模數計數則也需明示模數。

第22章 基礎圖演算法：詳解 （續）

原創驗算程式附錄: breadth_first_search.py

```
"""BFS distances on the course's layered example."""

from collections import deque

def bfs(graph, source):
    distance = {source: 0}
    queue = deque([source])
    while queue:
        u = queue.popleft()
        for v in graph.get(u, ()):
            if v not in distance:
                distance[v] = distance[u] + 1
                queue.append(v)
    return distance


graph = {"s": ["a", "b"], "a": ["c"], "b": ["c"], "c": []}
assert bfs(graph, "s") == {"s": 0, "a": 1, "b": 1, "c": 2}
print(bfs(graph, "s"))
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

第22章 基礎圖演算法：詳解 （續）

原創驗算程式附錄: graph_search_checks.py

```
"""Graph searches checked against independent all-pairs path recurrences."""
from collections import deque
from itertools import product
from math import inf

SAMPLE={'a':['b','c'],'b':['c'],'c':['a'],'d':['b','e'],'e':['d'],'f':['e']}


def transpose(graph):
    result={u:[] for u in graph}
    for u,neighbors in graph.items():
        for v in neighbors:result[v].append(u)
    return result


def bfs(graph,source):
    distance={u:inf for u in graph};parent={u:None for u in graph}
    ways={u:0 for u in graph};distance[source]=0;ways[source]=1
    queue=deque([source]);trace=[]
    while queue:
        u=queue.popleft()
        for v in graph[u]:
            if distance[v]==inf:
                distance[v]=distance[u]+1;parent[v]=u;queue.append(v)
            if distance[v]==distance[u]+1:ways[v]+=ways[u]
        trace.append((u,tuple(queue)))
    return distance,parent,ways,trace


def dfs(graph,order=None):
    colors={u:0 for u in graph};tin={};tout={};parents={u:None for u in graph}
    edges=[];finished=[];trees=[];white_reachable={};time=0
    def visit(u,tree):
        nonlocal time
        # Snapshot before u changes from white; independent reachability search.
        white={v for v in graph if colors[v]==0};seen={u};pending=[u]
        while pending:
            x=pending.pop()
            for y in graph[x]:
                if y in white and y not in seen:seen.add(y);pending.append(y)
        white_reachable[u]=seen
        time+=1;tin[u]=time;colors[u]=1;tree.append(u)
        for v in graph[u]:
            if colors[v]==0:
                edges.append((u,v,'T'));parents[v]=u;visit(v,tree)
            elif colors[v]==1:edges.append((u,v,'B'))
```

第22章 基礎圖演算法：詳解 （續）

```
            else:edges.append((u,v,'F' if tin[u]<tin[v] else 'C'))
        colors[u]=2;time+=1;tout[u]=time;finished.append(u)
    for u in graph if order is None else order:
        if colors[u]==0:
            tree=[];visit(u,tree);trees.append(tree)
    return dict(tin=tin,tout=tout,parent=parents,edges=edges,
                finished=finished,trees=trees,white=white_reachable)


def scc(graph,first=None):
    first=dfs(graph) if first is None else first
    return dfs(transpose(graph),reversed(first['finished']))['trees']


def kahn(graph):
    incoming={u:0 for u in graph}
    for neighbors in graph.values():
        for v in neighbors:incoming[v]+=1
    queue=deque(u for u in graph if incoming[u]==0);order=[]
    while queue:
        u=queue.popleft();order.append(u)
        for v in graph[u]:
            incoming[v]-=1
            if incoming[v]==0:queue.append(v)
    return order if len(order)==len(graph) else None


def all_pairs(graph):
    # Floyd-style path recurrence, including self-loop cycle detection separately.
    vertices=list(graph);distance={(u,v):0 if u==v else inf for u in vertices for v in
vertices}
    reach={(u,v):v in graph[u] for u in vertices for v in vertices}
    for u in vertices:
        for v in graph[u]:distance[u,v]=min(distance[u,v],1)
    for k in vertices:
        for u in vertices:
            for v in vertices:
                distance[u,v]=min(distance[u,v],distance[u,k]+distance[k,v])
                reach[u,v]=reach[u,v] or (reach[u,k] and reach[k,v])
    return distance,reach


def verify_graph(graph):
    vertices=list(graph);distance,reach=all_pairs(graph);first=dfs(graph)
    tin,tout=first['tin'],first['tout']
    assert sorted([*tin.values(),*tout.values()])==list(range(1,2*len(graph)+1))
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

第22章 基礎圖演算法：詳解 （續）

```
    assert {u:frozenset(v for v in vertices if tin[u]<=tin[v] and tout[v]<=tout[u])
            for u in vertices}=={u:frozenset(v) for u,v in first['white'].items()}
    for u in vertices:
        for v in vertices:
            if u==v:continue
            assert (tout[u]<tin[v] or tout[v]<tin[u]
                    or tin[u]<tin[v]<tout[v]<tout[u]
                    or tin[v]<tin[u]<tout[u]<tout[v])
    for u,v,kind in first['edges']:
        if kind=='T':assert first['parent'][v]==u and tin[u]<tin[v]<tout[v]<tout[u]
        elif kind=='F':assert first['parent'][v]!=u and tin[u]<tin[v]<tout[v]<tout[u]
        elif kind=='B':assert u==v or tin[v]<tin[u]<tout[u]<tout[v]
        else:assert tout[v]<tin[u]
    cyclic=any(reach[u,u] for u in vertices)
    assert cyclic==any(kind=='B' for _,_,kind in first['edges'])
    order=kahn(graph);assert (order is None)==cyclic
    if not cyclic:
        for candidate in (order,list(reversed(first['finished']))):
            positions={u:i for i,u in enumerate(candidate)}
            assert all(positions[u]<positions[v] for u in vertices for v in graph[u])
    groups=scc(graph,first);membership={u:i for i,group in enumerate(groups) for u in group}
    assert len(membership)==len(graph)
    for u in vertices:
        for v in vertices:
            assert (membership[u]==membership[v])==(u==v or (reach[u,v] and reach[v,u]))
    finishmax={i:max(tout[u] for u in group) for i,group in enumerate(groups)}
    for u in vertices:
        for v in graph[u]:
            if membership[u]!=membership[v]:
                assert finishmax[membership[u]]>finishmax[membership[v]]
    for source in vertices:
        computed,parent,ways,trace=bfs(graph,source)
        for v in vertices:
            assert computed[v]==distance[source,v]
            if v!=source and computed[v]<inf:
                assert v in graph[parent[v]] and computed[parent[v]]+1==computed[v]
            # Enumerate simple paths independently; shortest unit-edge paths are simple.
            count=0
            def paths(u,seen,length):
                nonlocal count
                if length>distance[source,v]:return
                if u==v:
                    if length==distance[source,v]:count+=1
                    return
                for w in graph[u]:
                    if w not in seen:paths(w,seen|{w},length+1)
```

第22章 基礎圖演算法：詳解 （續）

```
            paths(source,{source},0)
            assert ways[v]==count
        for _,queue in trace:
            values=[computed[u] for u in queue]
            assert values==sorted(values)
            assert not values or values[-1]<=values[0]+1


def main():
    total=0;sources=0
    for n in range(5):
        for mask in range(1<<(n*n)):
            graph={u:[v for v in range(n) if mask>>(n*u+v)&1] for u in range(n)}
            verify_graph(graph);total+=1;sources+=n
    first=dfs(SAMPLE)
    assert first['tin']==dict(a=1,b=2,c=3,d=7,e=8,f=11)
    assert first['tout']==dict(a=6,b=5,c=4,d=10,e=9,f=12)
    assert first['edges']==[('a','b','T'),('b','c','T'),('c','a','B'),('a','c','F'),
                            ('d','b','C'),('d','e','T'),('e','d','B'),('f','e','C')]
    assert scc(SAMPLE)==[['f'],['d','e'],['a','c','b']]
    d,p,_,trace=bfs(SAMPLE,'d')
    assert d==dict(a=3,b=1,c=2,d=0,e=1,f=inf)
    assert trace==[('d',('b','e')),('b',('e','c')),('e',('c',)),('c',('a',)),('a',())]
    diamond={0:[1,2],1:[3,4],2:[3,4],3:[5],4:[5],5:[]}
    assert list(bfs(diamond,0)[2].values())==[1,1,1,2,2,4]
    assert kahn({'a':['c'],'b':['c'],'c':['d'],'d':[]})==['a','b','c','d']
    assert kahn({'a':['c'],'b':['c'],'c':['d'],'d':['b']}) is None
    # The wrong root order really merges distinct SCCs in the transpose.
    tiny={'x':['y'],'y':[]}
    assert dfs(transpose(tiny),['y','x'])['trees']==[['y','x']]
    assert scc(tiny)==[['x'],['y']]
    # A finished reachable vertex need not be a descendant.
    wrong=dfs({'a':['b','c'],'b':[],'c':['b']})
    assert wrong['tout']['b']<wrong['tin']['c'] and 'b' not in wrong['white']['c']
    # Reversing every adjacency order leaves all invariant checks intact.
    verify_graph({u:list(reversed(v)) for u,v in SAMPLE.items()})
    print(f'PASS directed_graphs_with_self_loops_n0_to4={total} BFS_sources={sources} '
          'distance=all_pairs_oracle shortest_counts=simple_path_oracle '
          'SCC=mutual_reachability DFS=white_paths_and_intervals topo=cycle_equivalence '
          'sample_all8_edge_classes_and_two_pass_trace=True')


if __name__=='__main__':main()
```

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study